Assembly Monitoring System

Complete Line-by-Line Code Walkthrough

Every line explained for absolute beginners

How to Use This Guide

This document explains **every single line** of the assembly monitoring code. Red text shows the actual code, green text explains what it does. Even with zero programming experience, you'll understand this system by the end.

The system uses AI (YOLO) to watch a video camera and detect when assembly workers install parts. It checks that parts are installed in the correct order and displays progress on a web page.

SECTION 1: IMPORTING LIBRARIES (Lines 1-20)

These lines load external tools (libraries) that the program needs. Think of them as importing tools into a workshop - you can't build without your tools!

```
import time
```

What: Loads Python's time library
Why: Handles anything time-related - delays, timestamps, measuring duration
Example use: Measuring how long the assembly takes, ensuring 30-second wait after pin installation

```
import sys
```

What: Loads system library
Why: Access system functions, modify Python's import paths
In this code: Partially used (has commented code for path modification)

```
from uuid import uuid4
```

What: Imports the uuid4 function
Why: Generates unique random IDs
Example use: Creates unique filename for each recording: 'a7b2c8-1234-5678.mp4'

```
import queue
```

What: Loads queue data structure
Why: Thread-safe way to pass data between parts of program running simultaneously
Analogy: Like a line at a store - first in, first out

```
import threading
```

What: Loads Python's threading library
Why: Run multiple tasks simultaneously
Example: One thread captures video, another processes frames - both run at same time

```
import cv2
```

What: Loads OpenCV (cv2 = Computer Vision 2)
Why: THE main library for working with images and video
Can do: Read videos, draw shapes, add text, resize images, etc.

```
from skimage.metrics import structural_similarity as ssim
```

What: Imports SSIM function from scikit-image
Why: Compares how similar two images are (1.0=identical, 0.0=different)
Note: Imported but not actively used in current version

```
import av
```

What: Loads PyAV (Python + FFmpeg)
Why: Faster video processing than OpenCV, supports GPU acceleration
Usage: Reads RTSP camera stream efficiently

```
import math
```

What: Loads math functions
Why: Provides sqrt, sin, cos, and importantly math.dist()
Usage: Calculates distance between two hand positions

```
from collections import deque
```

What: Imports deque (double-ended queue)
Why: Special list with max size - automatically removes oldest when full
Example: deque(maxlen=30) keeps only last 30 detections

```
from flask import Flask, Response, request, url_for
```

What: Imports Flask web framework components
Why: Turns Python program into a web server
Components: Flask(app), Response(custom responses), request(incoming data), url_for(URLs)

```
import os
```

What: Operating system interface
Why: File operations, environment variables
Usage: Sets OpenCV logging levels via environment variables

```
import numpy as np
```

What: NumPy - numerical computing library
Why: Images ARE numpy arrays! A 1920×1080 image = array[1080, 1920, 3]
Essential for: All image/video processing

```
from ultralytics import YOLO
import ultralytics
```

What: Imports YOLO AI framework
Why: The 'brain' that detects objects in images
Trained to recognize: Assembly parts, tools, hand positions

```
ultralytics.checks()
```

What: Runs YOLO system check
Why: Verifies PyTorch, CUDA, dependencies are installed
Prints: System report showing what's available

SECTION 2: OPENCV CONFIGURATION (Lines 18-23)

```
try:
cv2.utils.logging.setLogLevel(cv2.utils.logging.LOG_LEVEL_ERROR)
```

What: Try to silence OpenCV warnings (modern version)
try block: Attempts code that might fail
Why: OpenCV prints LOTS of messages - we only want errors

```
except Exception:
os.environ['OPENCV_LOG_LEVEL'] = 'ERROR'
os.environ['OPENCV_FFMPEG_CAPTURE_OPTIONS'] = 'rtsp_transport;tcp'
```

What: If try failed, use environment variables (older OpenCV)
except Exception: Catches any error from try block
OPENCV_LOG_LEVEL: Only show errors
OPENCV_FFMPEG_CAPTURE_OPTIONS: Use TCP for RTSP (more reliable than UDP)

SECTION 3: LOADING AI MODELS (Lines 25-31)

```
pos_model = YOLO('models/yolov8n-pose.pt', task='pose',)
```

What: Load pose detection AI model
pos_model = Variable to store the model
'models/yolov8n-pose.pt': File containing trained weights
task='pose': Detects human body positions/keypoints
Detects: Hand positions, elbows, shoulders for tracking worker movements

```
pos_model.predict('defect_scanner_logo.png', conf=0.4, iou=0.8, imgsz=640, verbose=False, device=[0])
```

What: Warm-up prediction (makes future predictions faster)
'defect_scanner_logo.png': Dummy image (doesn't matter what)
conf=0.4: Confidence threshold - only show detections >40% confident
iou=0.8: Intersection-over-Union for overlapping detections
imgsz=640: Resize to 640×640 for processing
verbose=False: Don't print details
device=[0]: Use first GPU
Why warm-up: First prediction loads model into GPU memory (slow). Next predictions fast!

```
tower_model = YOLO('models/AL_tower_271225.pt', task='detect')
```

What: Load custom object detection model
'AL_tower_271225.pt': Custom model trained Dec 27, 2025
task='detect': Object detection (finding and labeling parts)
Trained on: End covers, bolts, clamps, hammers, drills, hands - all assembly components

```
print(tower_model.names)
```

What: Prints object class names
Outputs: {0: 'end_cover', 1: 'end_cover_bolt', 2: 'tower', ...}
Why useful: Confirms model loaded correctly, shows what it can detect

SECTION 4: GLOBAL VARIABLES (Lines 33-65)

```
all_steps = ['End cover fitment', 'End cover bolt fitment', 'Clamping', ...]
```

What: List of all 16 assembly step names
Why list: Each step has index 0-15
Usage: When step 5 completes, look up all_steps[5] = 'Circlip fitment'
Full list: 0-End cover, 1-Bolt, 2-Clamp, 3-Interlock pin, 4-Hammering, 5-Circlip, 6-Lock circlip, 7-Drain plug, 8-Loctite, 9-Cutoff valve, 10/11/12-Tightening, 13-Detent plunger, 14-Yellow spring, 15-Pink spring

```
cycle_check = []
cycle_check_time = []
```

What: Empty lists to track progress
cycle_check: Stores completed step numbers [0, 1, 2, 3]
cycle_check_time: Would store timestamps (not used currently)
Why empty: No steps completed when program starts

```
loc = threading.Lock()
```

What: Creates a thread lock
Why: Prevents multiple threads from accessing same data simultaneously
Analogy: Bathroom lock - only one person at a time
Usage: loc.acquire() = lock door, loc.release() = unlock door

```
dummy_img = cv2.imread('defect_scanner_logo.png')
logo = cv2.imread('defect-scanner-logo-transparent-cropped.png')
```

What: Load images from files
dummy_img: Placeholder shown before camera connects
logo: Logo overlaid on every frame (watermark)
cv2.imread(): Reads image file into numpy array

```
frame_ori = dummy_img.copy()
capFrame = dummy_img.copy()
```

What: Create copies of placeholder image
.copy(): Creates separate copy (not just reference)
frame_ori: Original frame before processing
capFrame: Current frame from camera (starts as placeholder)

```
rec_vid = False
video_rec = 0
```

What: Recording control variables
rec_vid: True=recording, False=not recording
video_rec: VideoWriter object (0=none, object=active recorder)
Toggle: Visit /record URL to switch between recording states

```
cycle_check_f = 0
```

What: Cycle status flag
Values: 0=in progress, 10=completed
Display: Shows 'In Progress' or 'Completed' status

```
cycle_start_time = None
```

What: When assembly cycle started
None: Python for 'no value yet'
Set to: time.time() when first step detected
Usage: Calculate elapsed time: time.time() - cycle_start_time

```
cls_ins = deque(maxlen=30)
tower_conf = deque(maxlen=20)
```

What: Deques for recent detection history
cls_ins: Last 30 object detections
tower_conf: Last 20 tower presence checks
Auto-remove: When 31st added, oldest drops off
Confirmation: Object must appear 3+ times in last 30 to be valid

```
sequence_break_f = 0
```

What: Sequence error tracker
Values: 0=no error, N=break at step N
Example: If circlip installed before hammering, sequence_break_f=5
Display: Shows blinking red warning when non-zero

```
detent_box_pos = [41, 232, 142, 309]
yellowS_box_pos = [216, 110, 279, 168]
pinkS_box_pos = [205, 187, 271, 238]
```

What: Screen regions for hand detection
Format: [x1, y1, x2, y2] = top-left to bottom-right
detent_box_pos: Where hands should be for detent plunger
yellowS_box_pos: Yellow spring installation area
pinkS_box_pos: Pink spring installation area
Check: if box[0] < hand_x < box[2] and box[1] < hand_y < box[3]: hand is inside!

```
hand_f = 0
spring_plungerf = 0
```

What: Hand detection flags
hand_f: Counter 0-3, increments when hand in position, decrements when not
spring_plungerf: 0=not verified, 1=installation confirmed
Verification: Both hands positioned correctly at proper distance

```
_, buffer = cv2.imencode('.jpg', dummy_img)
```

What: Encode image as JPEG
cv2.imencode(): Converts numpy array to JPEG bytes
Returns: (success_flag, jpeg_bytes)
_: Ignore success flag (underscore = don't care)
buffer: JPEG bytes ready to send to web browser
Why: Browsers can't display numpy arrays, need JPEG format

```
app = Flask(__name__,)
```

What: Create Flask web application
Flask(__name__): Initialize Flask app
__name__: Current module name
Enables: @app.route() decorators and app.run() server
Result: Python program becomes web server!

SECTION 5: CUSTOM THREAD CLASS (Lines 68-73)

```
class ThreadWithResult(threading.Thread):
```

What: Custom thread class that can return values
Why: Normal threads can't return values - this adds that feature
Inheritance: Extends threading.Thread with our additions

SECTION 6: FLASK ROUTES

Web endpoints users can visit

```
@app.route('/')
def index():
return 'Ashok Leyland'
```

Root URL - just shows company name

```
@app.route('/clear')
def cls():
cycle_check.clear()
cycle_check_f = 0
```

Resets assembly progress - visit between units

```
@app.route('/record')
def record():
```

Toggles video recording on/off

SECTION 7: MAIN DETECTION LOGIC (frameInf function)

This is the 'brain' - processes each frame and detects assembly steps

Key Detection Rules:

• **Tower presence:** Must detect tower 3+ times before allowing any steps

• **End cover (0):** class 0 detected 3+ times

• **Bolt (1):** class 1 detected 3+ times AND step 0 complete

• **Clamp (2):** class 3 detected 3+ times

• **Interlock pin (3):** class 5 detected 3+ times, starts 30-second timer

• **Hammering (4):** class 4 (hammer) 2+ times AND must follow step 3

• **Circlip (5):** class 6 detected 3+ times AND 30 seconds after step 3

• **Lock circlip (6):** hammer after circlip

• **Drain plug (7):** class 7 detected 3+ times

• **Loctite (8):** class 8 detected 3+ times AND step 7 complete

• **Cutoff valve (9):** class 9 detected 3+ times

• **Tightening (10-12):** Drill position in specific coordinates

• **Springs (13-15):** Hand positions in specific boxes

SECTION 8: VIDEO CAPTURE (getF function)

```
video_path = 'rtsp://sourab:tvsm123!@192.168.0.101/stream1?tcp'
```

What: Camera URL
Format: rtsp://username:password@ip/stream
?tcp: Use TCP transport
Credentials: sourab / tvsm123!

```
container = av.open(video_path, options={'rtsp_transport': 'tcp', 'hwaccel': 'cuda'})
```

What: Open video stream with PyAV
rtsp_transport=tcp: Reliable connection
hwaccel=cuda: Use GPU to decode video (MUCH faster)
container: Video stream object

```
for frame in container.decode(video=0):
frame = frame.to_ndarray(format='bgr24')
frame = frame[450:1500, 550:2000]
```

What: Process each video frame
decode(video=0): Decode video stream
to_ndarray(): Convert to numpy array in BGR format
[450:1500, 550:2000]: Crop to region of interest (y:y, x:x)
Result: 1050×1450 pixel region focused on assembly area

SECTION 9: WEB STREAMING

```
def generate_frames():
while True:
res = frameInf(capFrame.copy())
_, buffer = cv2.imencode('.jpg', res)
yield (b'--frame\r\n' b'Content-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n')
```

What: Generator function for MJPEG streaming
while True: Infinite loop
frameInf(): Process frame (add overlays, detect objects)
imencode(): Convert to JPEG
yield: Return frame and keep function alive
Multipart response: Each frame separated by --frame boundary
Result: Continuous stream of JPEG images = video!

```
@app.route('/video_feed')
def video_feed():
return Response(generate_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')
```

What: Endpoint that streams video
@app.route('/video_feed'): URL to visit
Response(): Custom HTTP response
generate_frames(): Generator function
mimetype: Tells browser this is MJPEG stream
Access: http://server-ip:5000/video_feed shows live video

SECTION 10: PROGRAM STARTUP

```
if __name__ == '__main__':
threading.Thread(target=getF, daemon=True).start()
app.run(host='0.0.0.0', port=5000)
```

What: Program entry point
if __name__ == '__main__': Only run if this is main file (not imported)
threading.Thread(target=getF): Create thread running getF function
daemon=True: Thread dies when main program exits
.start(): Start the thread (begins video capture)
app.run(): Start Flask web server
host='0.0.0.0': Listen on all network interfaces (accessible from other computers)
port=5000: Use port 5000
Result: Video capture runs in background, web server accepts connections

PROGRAM FLOW SUMMARY

Understanding the complete execution:

1. Program starts → Imports libraries, loads AI models

2. Variables initialized → All flags and lists set to defaults

3. Video capture thread starts → Connects to camera, continuously grabs frames

4. Web server starts → Flask listens on port 5000

5. User visits /live → Browser requests video stream

6. For each frame:

• Capture frame from camera

• Run YOLO detection on frame

• Check which objects detected

• Verify assembly sequence

• Draw overlays (boxes, text, status)

• Encode as JPEG

• Send to browser

7. Assembly progress tracked → Steps added to cycle_check list

8. User can:

• View live feed at /live

• Toggle recording at /record

• Reset progress at /clear

9. Cycle completes → All 16 steps detected in correct order

10. Visit /clear → Ready for next assembly unit

QUICK REFERENCE

Important URLs:

Live view: http://[server-ip]:5000/live

Video stream: http://[server-ip]:5000/video_feed

Clear progress: http://[server-ip]:5000/clear

Toggle recording: http://[server-ip]:5000/record

Color Codes:

Green: Step completed successfully

Yellow/Orange: Current step in progress

Red: Sequence break (wrong order)

White on blinking red: Step causing error

Key Variables to Remember:

cycle_check: List of completed steps [0,1,2,3...]

cls_ins: Last 30 detections (3+ needed to confirm)

sequence_break_f: 0=OK, N=error at step N

spring_plungerf: 1=spring installation verified

rec_vid: True=recording, False=not

End of Complete Code Walkthrough